Lecture 1
Relational Model

CSC1022 Introduction to Databases

CSC1005 Data Science & Databases
2025/2026 Semester 2

Prof. Mark Roantree
Uttaran Bera
School of Computing
@ Dublin City University

Agenda

1. **Overview**
   - Characteristics of Relations
2. **Constraints and Schemas**
   - Entity and Referential Integrity
3. **Constraint Violations**

Learning Objectives

Goal: Understand the Relation (table), attributes, their types (domain) . . .

- We want to build a dataset from scratch . . .
- Each column must be of the same data type, drawn only from a set of allowable values
- When you have multiple datasets (tables), how do you link them?
- How do you maintain the quality of your data?
- Can you place Rules on your data?

Overview

- The relational model is based on the mathematical concept of a **relation**, which is physically represented as a **table**.
- In the relational model, relations are used to hold information about the objects to be represented in the database.
- Case Study Approach: accomodation availability in a medium sized city.

Relational Data Structure

Relation

A relation is a table with columns and rows.

Attribute

An attribute is a named column of a relation.

- A **relation** is represented as a two-dimensional table in which the rows of the table correspond to individual records and the table columns correspond to attributes.
- Attributes can appear in any order and the relation will still be the same relation, and therefore convey the same meaning.

Example

- For example, the information on branch offices is represented by the *Branch* relation with columns for attributes *branchNo* (the branch number), *street*, *city*, and *postcode*.
- Similarly, the information on staff is represented by the *Staff* relation, with columns for attributes *staffNo, fName, lName, position, sex, DOB, salary*, and *branchNo* (the number of the branch the staff member works at).
- In Figure 1, a column contains values of a single attribute: the *branchNo* columns contain only numbers of existing branch offices.

Sample Relations

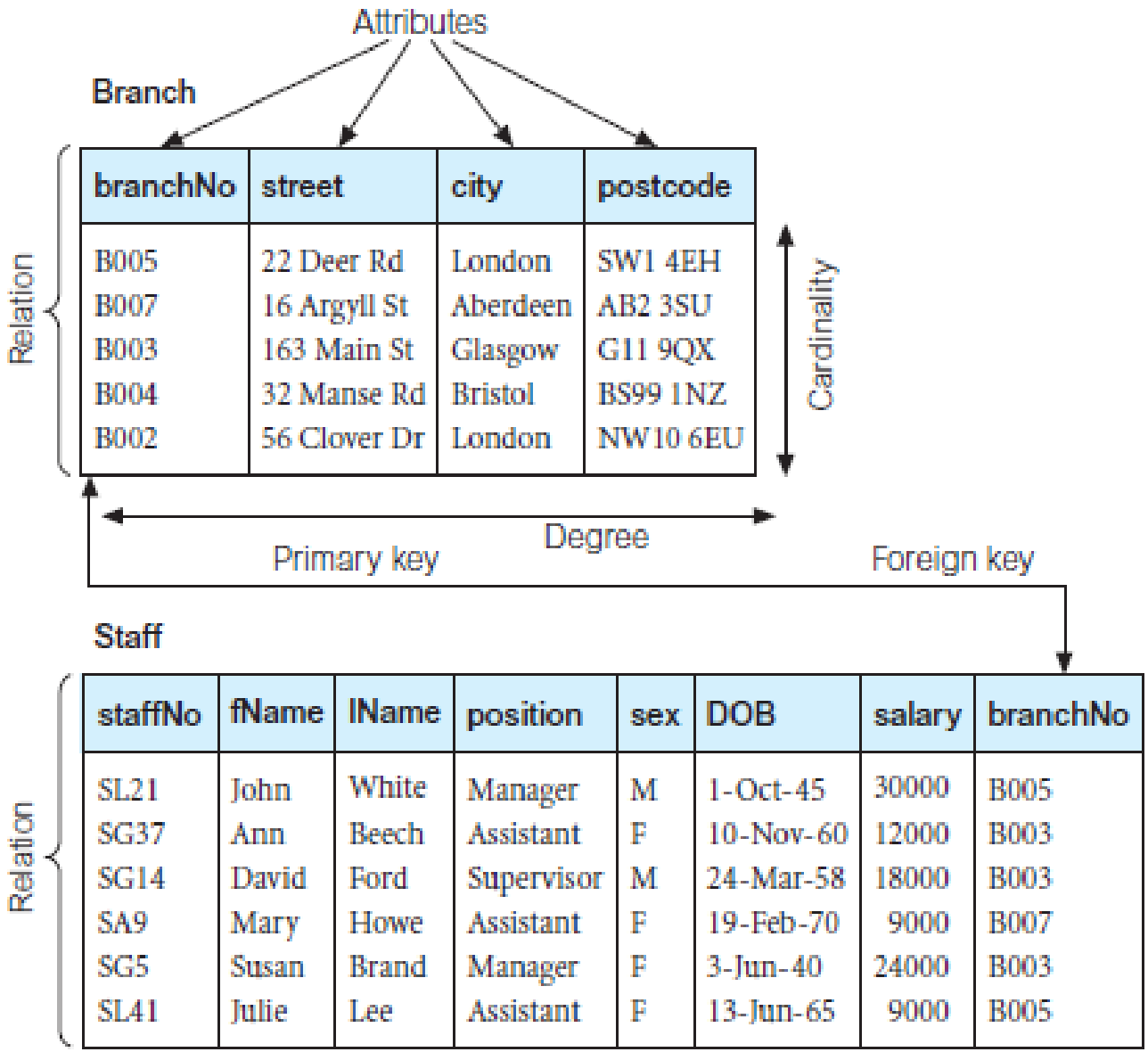

Figure 1: Branch and Staff Relations

Domains

Domain

A domain is the set of allowable values for one or more attributes.

- Every attribute in a relation is defined on a **domain**.
- Domains may be distinct for each attribute, or two or more attributes may be defined on the same domain.
- Generally, there will be values in a domain that do not currently appear as values in the attribute.

Sample Domains

Attribute	Domain Name	Meaning	Domain Definition
branchNo	BranchNumbers	The set of all possible branch numbers	character: size 4, range B001–B999
street	StreetNames	The set of all street names in Britain	character: size 25
city	CityNames	The set of all city names in Britain	character: size 15
postcode	Postcodes	The set of all postcodes in Britain	character: size 8
sex	Sex	The sex of a person	character: size 1, value M or F
DOB	DatesOfBirth	Possible values of staff birth dates	date, range from 1-Jan-20, format dd-mmm-yy
salary	Salaries	Possible values of staff salaries	monetary: 7 digits, range 6000.00–40000.00

Relational Data Structure

Tuple

A tuple is a row of a relation.

- The elements of a relation are the rows or **tuples** in the table.
  - In the *Branch* relation, each row contains four values, one for each attribute.
  - Tuples can appear in any order and the relation will still be the same relation, and therefore convey the same meaning.
- The structure of a relation, together with a specification of the domains and any other restrictions on possible values, is sometimes called its **intension**, which is usually fixed unless the meaning of a relation is changed to include additional attributes.
- The tuples are called the **extension** (or state) of a relation, which changes over time.

Degree of a Relation

A Relation's Degree

The degree of a relation is the number of attributes it contains.

- The *Branch* relation in Figure 1 has four attributes or **degree** four.
- This means that each row of the table is a *four-tuple*, containing four values.
- A relation with only one attribute would have degree one and be called a *unary* relation or *one-tuple*.
- A relation with two attributes is called *binary*, one with three attributes is called *ternary*, and after that the *n*-ary is usually used.
- The **degree** of a relation is a property of the **intension** of the relation.

A Relation's Cardinality

Cardinality

The cardinality of a relation is the number of tuples it contains.

- The number of tuples is called the **cardinality** of the relation and this changes as tuples are added or deleted.
- The **cardinality** is a property of the **extension** of the relation and is determined from the particular instance of the relation at any given moment.

Mathematical Relations

- Suppose that we have two sets, D_1 and D_2, where $D_1 = \{2, 4\}$ and $D_2 = \{1, 3, 5\}$.
- The Cartesian product of these two sets, written $D_1 \times D_2$, is the set of all ordered pairs such that the first element is a member of D_1 and the second element is a member of D_2.
- An alternative way is to find all combinations of elements from D_1 (first) and D_2.
- Here, we have:
  $D_1 \times D_2 = \{(2, 1), (2, 3), (2, 5), (4, 1), (4, 3), (4, 5)\}$

Cartesian Product Subsets

- Any subset of this Cartesian product is a relation.
- For example, we could produce a relation *R* such that: *R* = (2, 1), (4, 1)
- We can specify which ordered pairs will be in the relation by giving some condition for their selection.
- For example, if we observe that *R* includes all those ordered pairs in which the second element is 1, then we could write *R* as:

$$R = (x, y) \mid x \in D_1, y \in D_2, \text{ and } y = 1$$

Attributes

A relation schema R, denoted by $R(A_1, A_2, ..., A_n)$, is made up of a relation name R and a list of attributes $A_1, \ldots, A_n$.

- Each attribute A_i is the name of a role played by some domain D in the relation schema R.
- D is called the domain of A_i and is denoted by **dom(A_i)**.
- A relation schema is used to describe a relation; R is the name of this relation.
- The degree (or **arity**) of a relation is the number of attributes n of its relation schema.

Relations and Set Theory

Definition of a relation can be restated using set theory

- The Cartesian product specifies all possible combinations of values from the underlying domains.
- Hence, if we denote the total number of values, or cardinality, in a domain D by $|D|$ (assuming that all domains are finite), the total number of tuples in the Cartesian product is:

Relations and Set Theory

Definition of a relation can be restated using set theory

- The Cartesian product specifies all possible combinations of values from the underlying domains.
- Hence, if we denote the total number of values, or cardinality, in a domain D by $|D|$ (assuming that all domains are finite), the total number of tuples in the Cartesian product is:

$$|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times \ldots \times |\text{dom}(A_n)|$$

A Relation is not like a File!

- A relation is defined as a set of tuples.
- Mathematically, elements of a set have *no order* so tuples in a relation have no particular order.
- However, file records are physically stored on disk (or in memory), so there always is an order among the records.

A Relation is not like a File!

- A relation is defined as a set of tuples.
- Mathematically, elements of a set have *no order* so tuples in a relation have no particular order.
- However, file records are physically stored on disk (or in memory), so there always is an order among the records.
- This ordering indicates first, second, *n*th, and last records in the file.
- Similarly, when we display a relation as a table, the rows are displayed in a certain order.

Allowable Values

- Each value in a tuple is *atomic*: it is not divisible into components.
- Composite and multivalued attributes are not allowed.
- This model is sometimes called the flat relational model.
- Much of the theory behind the relational model was developed with this assumption in mind.
- It is called **the first normal form** assumption.

Allowable Values

- Each value in a tuple is *atomic*: it is not divisible into components.
- Composite and multivalued attributes are not allowed.
- This model is sometimes called the flat relational model.
- Much of the theory behind the relational model was developed with this assumption in mind.
- It is called **the first normal form** assumption.
- Hence, multivalued attributes must be represented by separate relations (eg. *address*).
- Composite attributes are represented only by their simple component attributes (eg. *address* as separate attributes).

NULL Values

- An important concept is that of NULL values, which are used to represent the values of attributes that may be unknown or may not apply to a tuple.
- A special value, called NULL, is used in these cases.
- Assume in Figure 1, that the DOB for SA9 was unknown, a NULL is recorded.
- In general, we can have several meanings for NULL values, such as value unknown, value exists but is not available, or attribute does not apply to this tuple (also known as value undefined).
- During database design, it is best to avoid NULL values as much as possible

Constraint Types

- There are various restrictions on data that can be specified on a relational database in the form of constraints.
- Constraints on databases can generally be divided into three main categories.
  - Constraints that are inherent in the data model: *inherent model-based* constraints or *implicit* constraints.
  - Constraints that can be directly expressed in schemas of the data model, typically by specifying them in the DDL (data definition language). We call these *schema-based* constraints or *explicit* constraints.
  - Constraints that cannot be directly expressed in the schemas and must be expressed and enforced by the application. We call these *application-based* or *semantic* constraints or *business rules*.

Constraint Types

- The characteristics of relations covered earlier are the inherent constraints of the relational model and belong to the first category. For example, the constraint that a relation cannot have duplicate tuples is an **inherent constraint**.
- The constraints we discuss now are of the second category: constraints expressed in the schema via the DDL.
- Constraints in the third category are difficult to express and enforce within the data model (they need program logic), so they are usually checked within the application programs that perform database updates.

Domain Constraints

- Domain constraints specify that within each tuple, the value of each attribute *A* must be an atomic value from the domain dom(*A*).
- The data types associated with domains typically include standard numeric data types for integers (such as short integer, integer, and long integer) and real numbers.
- Characters, Booleans, fixed-length strings, and variable-length strings are also available, as are date, time, timestamp, and money, or other special data types.

Key Constraints

- In the formal relational model, a relation is defined as a *set of tuples*.
- By definition, all elements of a set are distinct: all tuples in a relation must also be distinct.
- This means that no two tuples can have the same combination of values for all their attributes.
- Usually one (or multiple) attribute is chosen so that no two tuples may have the same value.

If SK represents this attribute(s), then for any two distinct tuples t_1 and t_2 in a relation state r of R, we have the constraint that: $t_1[SK] \neq t_2[SK]$

Key Constraints: superkey

- Any such set of attributes *SK* is called a **superkey** of the relation schema *R*.
- A superkey *SK* specifies a uniqueness constraint that no two distinct tuples in any state *r* of *R* can have the same value for *SK*.
- Every relation has at least one default superkey: the set of all its attributes.
- A superkey can have redundant attributes: so a more useful concept is that of a key with no redundancy.

A key *K* of a relation schema *R* is a superkey of *R* so that removing any attribute *A* from *K* leaves a set of attributes *K'* that is no longer a superkey of *R*.

Key Properties: Keys and Superkeys

- A key satisfies two properties
  1. Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key.
  2. It is a **minimal superkey**, if one cannot remove any attributes and still have the uniqueness constraint in condition 1 hold.

Key Properties: Keys and Superkeys

- A key satisfies two properties
  1. Two distinct tuples in any state of the relation cannot have identical values for (all) the attributes in the key.
  2. It is a **minimal superkey**, if one cannot remove any attributes and still have the uniqueness constraint in condition 1 hold.
- Whereas the first property applies to both **keys** and **superkeys**, the second property is required only for **keys**.
- Hence, *a key is also a superkey* but not vice versa.

Key Properties: Examples

- Consider the *Branch* relation in Figure 1.
- The attribute *branchNo* is a key of *Branch* because no two Branch tuples can have the same value for *branchNo*.
- Any *set of attributes* that includes *branchNo*, eg. {branchNo, street, city}, is a superkey.

Key Properties: Examples

- Consider the *Branch* relation in Figure 1.
- The attribute *branchNo* is a key of *Branch* because no two Branch tuples can have the same value for *branchNo*.
- Any *set of attributes* that includes *branchNo*, eg. {branchNo, street, city}, is a superkey.
- However, the superkey {branchNo, street, city}, is not a key of *Branch* because removing *street* or *city* or both from the set still leaves a superkey.
- In general, any superkey formed from a single attribute is also a key.
- A key with multiple attributes must require all its attributes together to have the uniqueness property.

Key Properties: Multiple Keys

- In general, a relation schema may have more than one key.
- In this case, each of the keys is called a **candidate key**.
- It is common to designate one of the candidate keys as the **primary key** of the relation.
- This is the candidate key whose values are used to identify tuples in the relation.

Primary Keys and Entity Integrity

Entity Integrity

The **entity integrity constraint** states that no primary key value can be NULL.

- This is because the primary key value is used to identify individual tuples in a relation.
- Having NULL values for the primary key implies that we cannot identify some tuples.
- For example, if two or more tuples had NULL for their primary keys, we may not be able to distinguish them if we try to reference them from other relations.
- Key constraints and entity integrity constraints are specified on individual relations.

Foreign Keys and Referential Integrity

Referential Integrity

The **referential integrity constraint** is specified between two relations and is used to maintain the consistency among tuples in the two relations.

- Informally, the referential integrity constraint states that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation.
- Can you see this constraint in figure Figure 1?

Referential Integrity Definition

- First we define the concept of a **foreign key**, a constraint between the two relations *R1* and *R2*.
- A set of attributes FK in relation *R1* is a foreign key of *R1* that references relation *R2* if it satisfies the rules:

Referential Integrity Definition

- First we define the concept of a **foreign key**, a constraint between the two relations *R1* and *R2*.
- A set of attributes FK in relation *R1* is a foreign key of *R1* that references relation *R2* if it satisfies the rules:
  1. The attributes in FK have the same domain(s) as the primary key attributes PK of *R2*: the FK references the relation *R2*.
  2. A value of FK in a tuple *t1* of the current state r1(*R1*) either occurs as a value of PK for some tuple *t2* in the current state r2(*R2*) or is NULL.
- In the former case, we have t1[FK] = t2[PK], and we say that the tuple t1 references or refers to the tuple t2.

DDL specifies constraints

- In this definition, *R1* is called the referencing relation and *R2* is the referenced relation.
- If these two conditions hold, a referential integrity constraint from *R1* to *R2* is said to hold.
- In a database of many relations, there are usually many referential integrity constraints.

Constraint Types (Summary)

Constraints on databases can generally be divided into three main categories.

- Constraints that are inherent in the data model: Domain constaints.
- Constraints that can be directly expressed in the DDL: Foreign key constraints.
- Constraints expressed and enforced by the application. Business rules eg. customer cannot be overdrawn in more than 1 account.

Constraint Violations

- There are three basic operations that can change the states of relations in the database: Insert, Delete, and Update.
- **Insert** is used to insert one or more new tuples in a relation; **Delete** is used to delete tuples; and **Update** is used to change the values of some attributes in existing tuples.
- Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated.
- We look at violating different types of constraints and the actions to take if an operation causes a violation.

INSERT Violations

INSERT Operation

The **Insert** operation provides a list of attribute values for a new tuple *t* that is to be inserted into a relation *R*.

- *Domain constraints* can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type.
- *Key constraints* can be violated if a key value in the new tuple *t* already exists in another tuple in the relation r(*R*).

INSERT Violations

INSERT Operation

The **Insert** operation provides a list of attribute values for a new tuple *t* that is to be inserted into a relation *R*.

- *Domain constraints* can be violated if an attribute value is given that does not appear in the corresponding domain or is not of the appropriate data type.
- *Key constraints* can be violated if a key value in the new tuple *t* already exists in another tuple in the relation r(*R*).
- *Entity integrity* can be violated if any part of the primary key of the new tuple *t* is NULL.
- *Referential integrity* can be violated if the value of any foreign key in *t* refers to a tuple that does not exist in the referenced relation.

Constraint Violation Examples

Provide an example for all 4 using the schema in figure 1

- If an insertion violates one or more constraints, the default option is to reject the insertion.

Constraint Violation Examples

Provide an example for all 4 using the schema in figure 1

- If an insertion violates one or more constraints, the default option is to reject the insertion.
- Domain example?

Constraint Violation Examples

Provide an example for all 4 using the schema in figure 1

- If an insertion violates one or more constraints, the default option is to reject the insertion.
- Domain example?
- Key constraint example?

Constraint Violation Examples

Provide an example for all 4 using the schema in figure 1

- If an insertion violates one or more constraints, the default option is to reject the insertion.
- Domain example?
- Key constraint example?
- Entity integrity?

Constraint Violation Examples

Provide an example for all 4 using the schema in figure 1

- If an insertion violates one or more constraints, the default option is to reject the insertion.
- Domain example?
- Key constraint example?
- Entity integrity?
- Referential integrity example?

DELETE Violations

- The **Delete** operation can violate only referential integrity.
- This occurs if the tuple being deleted is referenced by foreign keys from other tuples in the database. To fix:
- The first option, called `restrict`, is to reject the deletion.
- The second option, called `cascade`, is to attempt to cascade (or propagate) the deletion by deleting tuples that reference the tuple that is being deleted.

DELETE Violations: NULL

- The third option, called `set null` or `set default`, is to modify the referencing attribute values that cause the violation; each such value is either set to NULL or changed to reference another default valid tuple.
- Notice that if a referencing attribute that causes a violation is part of the primary key, it cannot be set to NULL; otherwise, it would violate entity integrity.

UPDATE Violations

- The **Update** (or Modify) operation is used to change the values of one or more attributes in a tuple (or tuples) of some relation *R*.
- It is necessary to specify a condition on the attributes of the relation to select the tuple (or tuples) to be modified.
- Updating an attribute that is neither part of a primary key nor of a foreign key usually causes no problems.
- Modifying a primary key value is similar to deleting one tuple and inserting another in its place because we use the primary key to identify tuples.

Lecture Ends